

# Самостійна робота №9: Соціальна аутентифікація - OAuth 2.0

**Тема:** Соціальна аутентифікація - OAuth 2.0

**Мета:** Зрозуміти протокол OAuth 2.0, вивчити процес аутентифікації через Google та GitHub за допомогою Passport.js, навчитися зберігати сесії користувачів.

**Кількість годин:** 3

## Теоретичний матеріал

### OAuth 2.0 - огляд протоколу

OAuth 2.0 - протокол авторизації, який дозволяє застосунку отримати обмежений доступ до ресурсів користувача на іншому сервісі без передачі пароля. Учасники процесу:

- **Resource Owner** - користувач, який надає доступ.
- **Client** - застосунок, що запитує доступ.
- **Authorization Server** - сервер, що видає токени (Google, GitHub).
- **Resource Server** - сервер з ресурсами користувача (Google API, GitHub API).

### Authorization Code Flow

Найбезпечніший і рекомендований flow для серверних застосунків:

1. Застосунок перенаправляє користувача на Authorization Server з параметрами `client_id`, `redirect_uri`, `scope`, `state`.
2. Користувач авторизується та надає дозвіл.
3. Authorization Server перенаправляє назад з `authorization_code`.
4. Серверна частина обмінює `code` на `access_token` (та опціонально `refresh_token`), надсилаючи `client_id` та `client_secret`.
5. Застосунок використовує `access_token` для доступу до ресурсів.

## Passport.js - стратегії OAuth

Passport.js - middleware для аутентифікації у Node.js/Express. Для OAuth 2.0 використовуються стратегії:

- **passport-google-oauth20** - аутентифікація через Google.
- **passport-github2** - аутентифікація через GitHub.

Кожна стратегія конфігурується з `clientID`, `clientSecret` та `callbackURL`. Після успішної авторизації стратегія отримує профіль користувача (email, ім'я, аватар).

## Реєстрація OAuth-застосунку

Перед використанням необхідно зареєструвати застосунок у провайдера: Google Cloud Console (APIs & Services > Credentials) або GitHub Settings > Developer settings > OAuth Apps. При реєстрації вказуються `redirect_uri` та отримуються `client_id` і `client_secret`.

## Збереження сесій

Після OAuth-аутентифікації користувача потрібно зберегти. Підходи: створення або оновлення запису в БД за `providerId` (наприклад, `googleId`), генерація власного JWT-токена, збереження сесії через `express-session` (серверні сесії). У Nest.js інтеграція з Passport відбувається через `@nestjs/passport` та AuthGuard.

## Контрольні питання

1. Що таке OAuth 2.0 і яку проблему він вирішує?
2. Які ролі визначає протокол OAuth 2.0 (Resource Owner, Client, Authorization Server, Resource Server)?
3. Опишіть кроки Authorization Code Flow.
4. Чому Authorization Code Flow безпечніший за Implicit Flow?
5. Як зареєструвати OAuth-застосунок у Google Cloud Console?
6. Яку роль виконує `client_secret` і чому його не можна передавати на клієнт?

7. Як зберегти користувача після OAuth-аутентифікації у базі даних?

### **Порядок виконання**

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.

### **Рекомендовані джерела**

- [OAuth 2.0 - RFC 6749](#) - специфікація протоколу
- [OAuth 2.0 Simplified](#) - зрозуміле пояснення OAuth 2.0
- [Passport.js - документація](#) - офіційна документація
- [Google OAuth 2.0 - документація](#) - гайд Google
- [GitHub OAuth - документація](#) - гайд GitHub
- [Nest.js - Authentication](#) - аутентифікація у Nest.js